

MRP-Projekt-Dokumentation

1. Projektdesign und Architektur-Entscheidungen

Technische Architektur

Das Projekt folgt einer klaren 3-Schichten-Architektur:

Controller-Schicht: UserController, MediaController - Verarbeiten HTTP-Requests

Service-Schicht: AuthService, MediaService, RatingService, etc. - Enthalten Geschäftslogik

Repository-Schicht: UserRepository, MediaRepository, etc. - Kümern sich um Datenpersistenz

Entscheidungen und Begründungen

1. Eigenimplementierung des HTTP-Servers

Entscheidung: Verwendung von `com.sun.net.httpserver.HttpServer` statt Frameworks wie Spring

Begründung: Die Aufgabenstellung verbietet Web-Frameworks. Die Low-Level-Implementierung ermöglicht tiefes Verständnis von HTTP-Protokoll und REST-Prinzipien

2. Token-basierte Authentifizierung

Entscheidung: Eigene Token-Implementierung mit In-Memory-Speicherung

Begründung: Einfachheit und Transparenz für Lernzwecke. Tokens werden als `username + "-mrpToken-" + UUID` generiert

3. Transaktionsbehandlung in Repositories

Entscheidung: Manuelle Transaktionssteuerung mit `setAutoCommit(false)` und `commit()/rollback()`

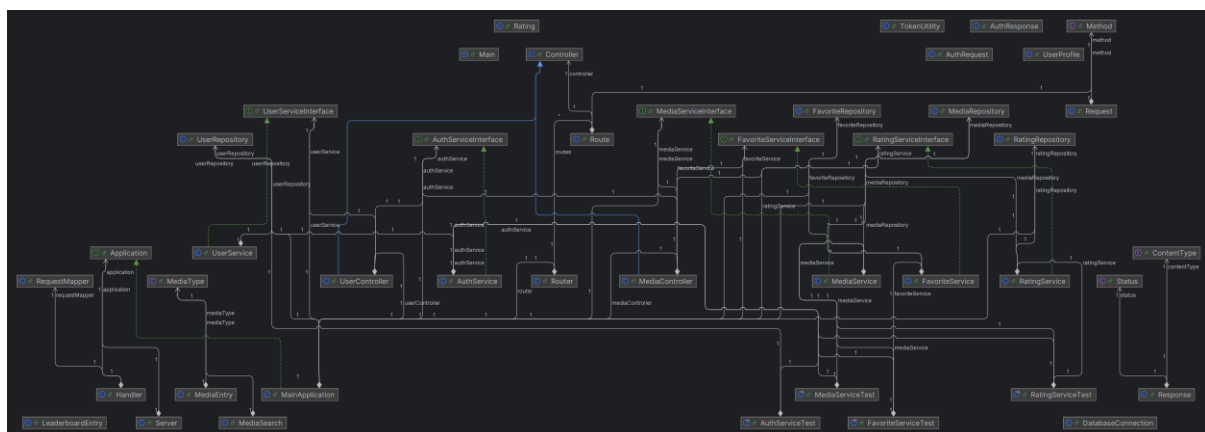
Begründung: Insbesondere bei `MediaRepository` notwendig, da `Media-Entries` und `Genres` atomar gespeichert werden müssen

4. Dependency Injection über Konstruktor

Entscheidung: Manuelle Dependency Injection ohne Framework

Begründung: Erhöht Testbarkeit und Entkopplung der Komponenten

2. Klassendiagramm (generiert von IntelliJ)



3. Schwierigkeiten und Lösungen

Schwierigkeit 1: Einhaltung der Application-Architektur

Problem: Anfängliche Vermischung von Controller- und Service-Logik führte zu unübersichtlichem Code.

Lösung:

Klare Trennung der Verantwortlichkeiten eingeführt

Controller: Nur Request/Response-Handling

Service: Reine Geschäftslogik

Repository: Nur Datenzugriff

Schwierigkeit 2: Korrektes Routen-Überprüfen

Problem: Der Router erkannte parametrisierte Pfade wie `/users/{username}/profile` nicht korrekt.

Lösung:

Implementierung von Regex-basiertem Path-Matching

`Router.matchesPath()` wandelt `{parameter}` in Regex-Gruppen um

Beispiel: `/users/{username}/profile` → `^/users/(?<username>[^\V]+)/profile$`

Schwierigkeit 3: Dependency Injection Implementierung

Problem: Manuelle Erstellung von Abhängigkeiten in Controllern führte zu starker Kopplung.

Lösung:

Konstruktor-Injection in allen Services und Controllern

MainApplication fungiert als Composition Root

4. Unit-Testing Strategie

Testabdeckung

Service-Schicht vollständig getestet: AuthServiceTest, MediaServiceTest, RatingServiceTest, FavoriteServiceTest

Testfokus: Geschäftslogik, nicht HTTP- oder Datenbank-Layer

Testphilosophie: Jede Service-Methode hat mindestens einen positiven und einen negativen Testfall

Warum diese Logik getestet wurde

Authentifizierung (AuthServiceTest): Kritisch für Sicherheit

Media-Validierung (MediaServiceTest): Kerngeschäft des Systems

Rating-Logik (RatingServiceTest): Komplexe Regeln (Duplikate, Bestätigungen)

Favoriten-Verwaltung (FavoriteServiceTest): Nutzerinteraktion sichern

5. SOLID-Prinzipien im Projekt

Open/Closed Principle - Beispiel: Controller-Erweiterung

Basisklasse Controller mit Hilfsmethoden (json(), text())

Neue Controller können erweitert werden ohne Basisklasse zu ändern

Beispiel: "UserController extends Controller"

Single Responsibility Principle – Beispiel: Repository-Klassen

Verantwortung: Nur Datenzugriff

NICHT verantwortlich: Geschäftslogik, Validierung, HTTP-Handling

Code-Ausschnitt:

```
public boolean createRating(Rating rating) { //usage new *
    String sql = "INSERT INTO ratings (id, media_id, username, stars, comment, comment_confirmed, likes) " +
        "VALUES (?, ?, ?, ?, ?, ?, ?)";

    try (Connection conn = DatabaseConnection.getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql)) {

        stmt.setString(1, rating.getId());
        stmt.setString(2, rating.getMediaId());
        stmt.setString(3, rating.getUsername());
        stmt.setInt(4, rating.getStars());
        stmt.setString(5, rating.getComment());
        stmt.setBoolean(6, rating.isCommentConfirmed());
        stmt.setInt(7, rating.getLikes());

        return stmt.executeUpdate() > 0;
    } catch (SQLException e) {
        e.printStackTrace();
        return false;
    }
}
```

```
public boolean isFavorite(String username, String mediaId) { //usage new *
    String sql = "SELECT 1 FROM favorites WHERE username = ? AND media_id = ?";

    try (Connection conn = DatabaseConnection.getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql)) {

        stmt.setString(1, username);
        stmt.setString(2, mediaId);
        ResultSet rs = stmt.executeQuery();

        return rs.next();
    } catch (SQLException e) {
        e.printStackTrace();
        return false;
    }
}
```